

Основы языка программирования Python¹

Объектно-ориентированное программирование

Объявление класса TextPost (textpost.py)

2

```
from datetime import datetime, timezone
```

```
class TextPost:
```

```
    """Класс текстового поста для блога"""
```

```
    def __init__(self, author, text):  
        print(f"TextPost.__init__(). self: {self}")  
        self.author = author  
        self.text = text  
        self.date = datetime.now(timezone.utc)
```

TextPost
author: str text: str date: datetime

Создание экземпляра класса TextPost (main.py)

3

```
from textpost import TextPost

if __name__ == "__main__":
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")

    print(f"{type(post)=}")
    print()
    print(dir(post))
    print()
    print("Автор:", post.author)
    print("Дата:", post.date)
```

```
> python main.py
```

```
TextPost.__init__(). self: <textpost.TextPost object at 0x7fa090f47380>
```

```
type(post)=<class 'textpost.TextPost'>
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__',  
 '__format__', '__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__', '__init__',  
 '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',  
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__',  
 '__subclasshook__', '__weakref__', 'author', 'date', 'text']
```

Автор: Толстой Л.Н.

Дата: 2025-03-23 07:36:39.146816+00:00

Не рекомендуемый способ добавления полей класса (main.py)

5

```
from textpost import TextPost
```

```
if __name__ == "__main__":  
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")
```

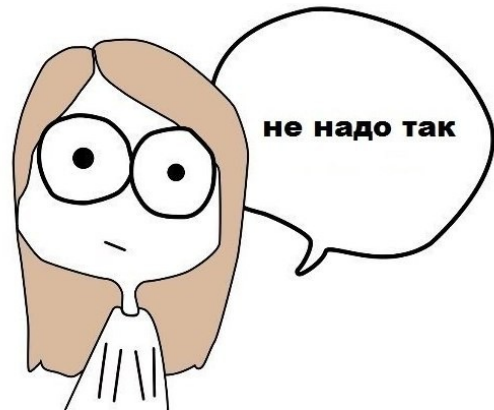
```
    print(dir(post))
```

```
    # Не рекомендуемый способ добавления полей класса
```

```
    post.title = "Война и мир"
```

```
    print()
```

```
    print(dir(post))
```



src/14. 00P/example_02/main.py

```
> python main.py
```

```
TextPost.__init__(). self: <textpost.TextPost object at 0x7f6174b374d0>
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__',  
 '__format__', '__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__', '__init__',  
 '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',  
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__',  
 '__subclasshook__', '__weakref__', 'author', 'date', 'text']
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__',  
 '__format__', '__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__', '__init__',  
 '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',  
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__',  
 '__subclasshook__', '__weakref__', 'author', 'date', 'text', 'title']
```

Изменение полей класса (main.py)

7

```
from textpost import TextPost
```

```
if __name__ == "__main__":  
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")  
  
    print("Автор:", post.author)  
    print("Дата:", post.date)  
    print("Текст:", post.text)  
  
    post.author = "Чехов А.П."  
    print()  
    print("Автор:", post.author)  
    print("Дата:", post.date)  
    print("Текст:", post.text)
```

```
> python main.py
```

```
TextPost.__init__(). self: <textpost.TextPost object at 0x7f32ac147380>
```

```
Автор: Толстой Л.Н.
```

```
Дата: 2025-03-23 07:53:58.041103+00:00
```

```
Текст: Очень длинный текст...
```

```
Автор: Чехов А.П.
```

```
Дата: 2025-03-23 07:53:58.041103+00:00
```

```
Текст: Очень длинный текст...
```


Соглашения об области видимости полей класса (textpost.py)

9

```
from datetime import datetime, timezone
```

```
class TextPost:
```

```
    """Класс текстового поста для блога"""
```

```
    def __init__(self, author, text):
        self._author = author
        self._text = text
        self._date = datetime.now(timezone.utc)
        self._save()
```

```
    def _save(self):
        print("Пост сохранен.")
```

```
    def get_author(self): return self._author
```

```
    def get_text(self): return self._text
```

```
    def set_text(self, value):
        self._text = value
        self._save()
```

```
    def get_date(self): return self._date
```

TextPost
<code>_author: str</code> <code>_text: str</code> <code>_date: datetime</code>
<code>_save(): None</code> <code>get_author(): str</code> <code>get_text(): str</code> <code>set_text(str): None</code> <code>get_date(): datetime</code>

src/14. 00P/example_04/textpost.py

```
from textpost import TextPost

if __name__ == "__main__":
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")

    print("Автор:", post.get_author())
    print("Дата:", post.get_date())

    print("Изменяем текст поста")
    post.set_text("Еще более длинный текст...")
```

```
> python main.py
```

Пост сохранен.

Автор: Толстой Л.Н.

Дата: 2025-03-23 08:21:07.757908+00:00

Изменяем текст поста

Пост сохранен.

Нарушение соглашения об области видимости полей класса (main.py)

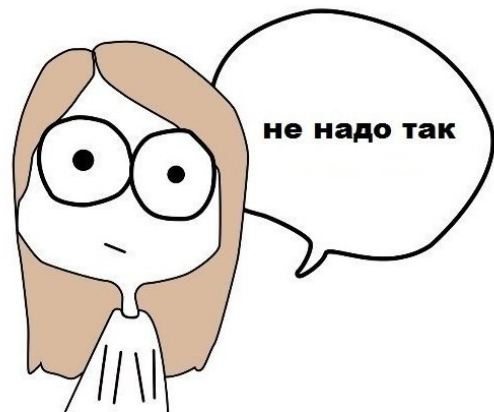
12

```
from textpost import TextPost

if __name__ == "__main__":
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")

    print("Автор:", post._author)
    print("Дата:", post._date)

    print("Изменяем текст поста")
    post._text = "Еще более длинный текст..."
```



src/14. OOP/example_05/main.py

```
> python main.py
```

Пост сохранен.

Автор: Толстой Л.Н.

Дата: 2025-03-23 08:26:03.064119+00:00

Изменяем текст поста

Свойства класса

Класс со свойствами (textpost.py)

15

```
class TextPost:
    """Класс текстового поста для блога"""

    def __init__(self, author, text):
        self._author = author
        self._text = text
        self._date = datetime.now(timezone.utc)
        self._save()

    def _save(self):
        print("Пост сохранен.")

    @property
    def author(self): return self._author

    @property
    def text(self): return self._text

    @text.setter
    def text(self, value):
        self._text = value
        self._save()

    @property
    def date(self): return self._date
```

TextPost
<code>_author: str</code> <code>_text: str</code> <code>_date: datetime</code>
<code>_save(): None</code> <code>author(): str</code> <code>text(): str</code> <code>text(str): None</code> <code>date(): datetime</code>

src/14. 00P/example_06/textpost.py

Использование свойств класса (main.py)

```
from textpost import TextPost

if __name__ == "__main__":
    post = TextPost("Толстой Л.Н.", "Очень длинный текст...")

    print("Автор:", post.author)
    print("Дата:", post.date)

    print("Изменяем текст поста")
    post.text = "Еще более длинный текст..."
```



```
> python main.py
```

Пост сохранен.

Автор: Толстой Л.Н.

Дата: 2025-03-23 08:42:14.245331+00:00

Изменяем текст поста

Пост сохранен.

Наследование

UML-диаграмма классов без использования наследования

19

TextPost
<code>_author: str</code> <code>_text: str</code> <code>_date: datetime</code>
<code>author(): str</code> <code>date(): datetime</code> <code>text(): str</code> <code>text(str): None</code> <code>format(): str</code>

ImagePost
<code>_author: str</code> <code>_image: str</code> <code>_date: datetime</code>
<code>author(): str</code> <code>date(): datetime</code> <code>image(): str</code> <code>image(str): None</code> <code>format(): str</code>

Использование классов TextPost и ImagePost (main.py)

20

```
from textpost import TextPost
from imagepost import ImagePost

post1 = TextPost("Толстой Л.Н.", "Очень длинный текст...")
post2 = ImagePost("Малевич К.С.", "black_square.jpg")

print(post1.format())
print()
print(post2.format())
```

```
> python main.py
```

Текстовый пост сохранен.

Пост с картинкой сохранен.

=====

Автор: Толстой Л.Н.

Дата: 2025-03-23 09:30:27.403127+00:00

Очень длинный текст...

=====

=====

Автор: Малевич К.С.

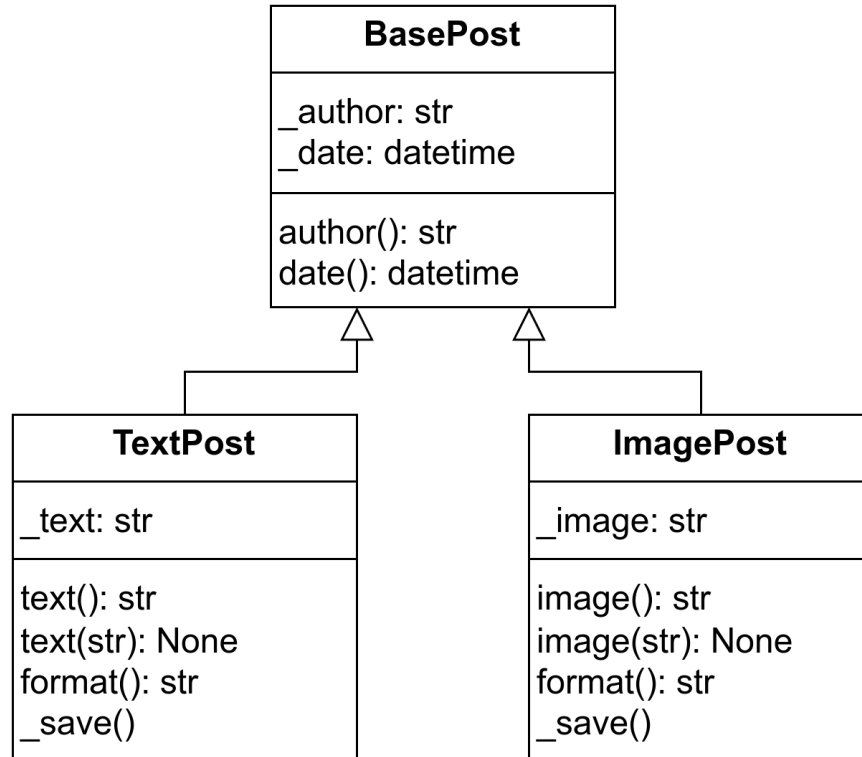
Дата: 2025-03-23 09:30:27.403145+00:00

Картинка: black_square.jpg

=====

UML-диаграмма классов с использованием базового класса

22



Базовый класс BasePost

```
from datetime import datetime, timezone

class BasePost:
    """Базовый класс для постов блога"""

    def __init__(self, author):
        print("BasePost.__init__()")
        self._author = author
        self._date = datetime.now(timezone.utc)

    @property
    def author(self): return self._author

    @property
    def date(self): return self._date
```

Конструкторы производных классов

```
class TextPost(BasePost):  
    """Класс текстового поста для блога"""  
  
    def __init__(self, author, text):  
        print("TextPost.__init__()")  
        super().__init__(author)  
        self._text = text  
        self._save()  
    ...  
  
class ImagePost(BasePost):  
    """Класс поста с картинкой для блога"""  
  
    def __init__(self, author, image):  
        print("TextPost.__init__()")  
        super().__init__(author)  
        self._image = image  
        self._save()  
    ...
```


Использование классов TextPost и ImagePost (main.py)

25

```
from textpost import TextPost
from imagepost import ImagePost

post1 = TextPost("Толстой Л.Н.", "Очень длинный текст...")
post2 = ImagePost("Малевич К.С.", "black_square.jpg")

print(post1.format())
print()
print(post2.format())
```

```
> python main.py
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Текстовый пост сохранен.
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Пост с картинкой сохранен.
```

```
=====
```

```
Автор: Толстой Л.Н.
```

```
Дата: 2025-03-23 09:44:55.251337+00:00
```

```
----
```

```
Очень длинный текст...
```

```
=====
```

```
=====
```

```
Автор: Малевич К.С.
```

```
Дата: 2025-03-23 09:44:55.251371+00:00
```

```
Картинка: black_square.jpg
```

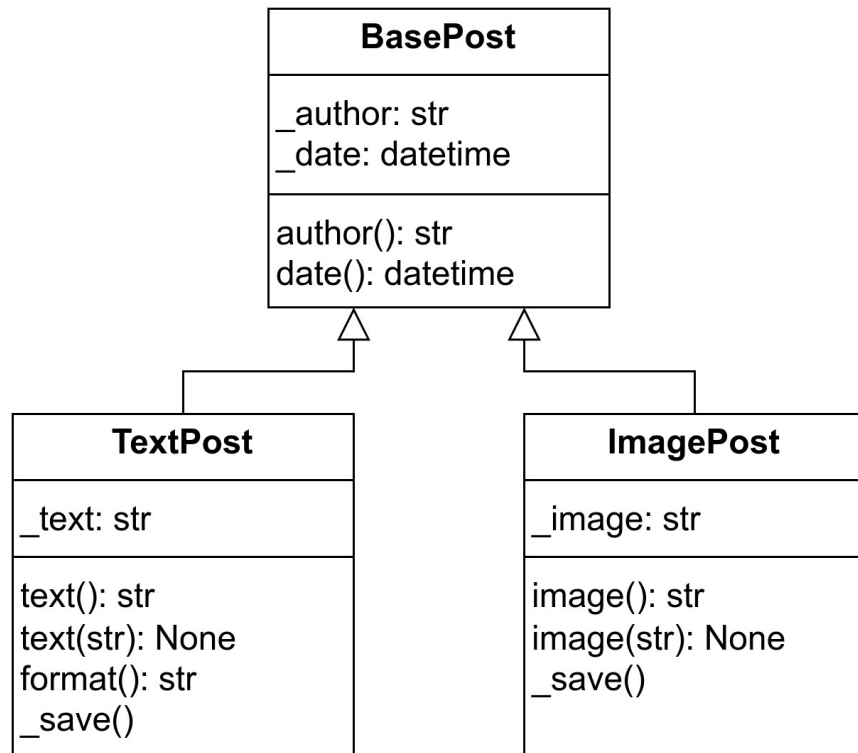
```
=====
```

```
src/13. 00P/example_08/
```

Абстрактные базовые классы

Пример с ошибкой.

Отсутствует ожидаемый метод



Пример с ошибкой.

Отсутствует ожидаемый метод

29

```
from textpost import TextPost
from imagepost import ImagePost

feed = []
feed.append(TextPost("Толстой Л.Н.", "Очень длинный текст..."))
feed.append(ImagePost("Малевич К.С.", "black_square.jpg"))

for post in feed:
    print(post.format())
```

```
> python main.py
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Текстовый пост сохранен.
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Пост с картинкой сохранен.
```

```
=====
```

```
Автор: Толстой Л.Н.
```

```
Дата: 2025-03-23 09:53:35.771003+00:00
```

```
----
```

```
Очень длинный текст...
```

```
=====
```

```
Traceback (most recent call last):
```

```
File "../main.py", line 9, in <module>
```

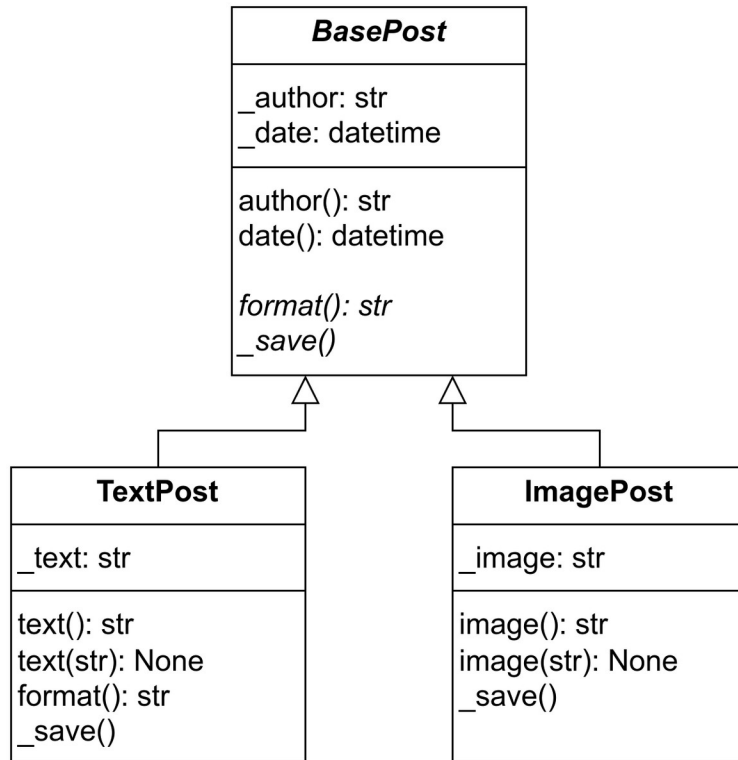
```
    print(post.format())
```

```
    ^^^^^^^^^^^^^
```

```
AttributeError: 'ImagePost' object has no attribute 'format'
```

```
src/14. 00P/example_09/
```

Диаграмма с абстрактным базовым классом `BasePost` и отсутствующим методом `format()` в классе `ImagePost`



Абстрактный базовый класс BasePost

```
from abc import ABCMeta, abstractmethod
from datetime import datetime, timezone

class BasePost(metaclass=ABCMeta):
    """Базовый класс для постов блога"""

    def __init__(self, author):
        print("BasePost.__init__()")
        self._author = author
        self._date = datetime.now(timezone.utc)

    @property
    def author(self): return self._author

    @property
    def date(self): return self._date

    @abstractmethod
    def format(self):
        ...

    @abstractmethod
    def _save(self):
        ...
```



```
> python main.py
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Текстовый пост сохранен.
```

```
Traceback (most recent call last):
```

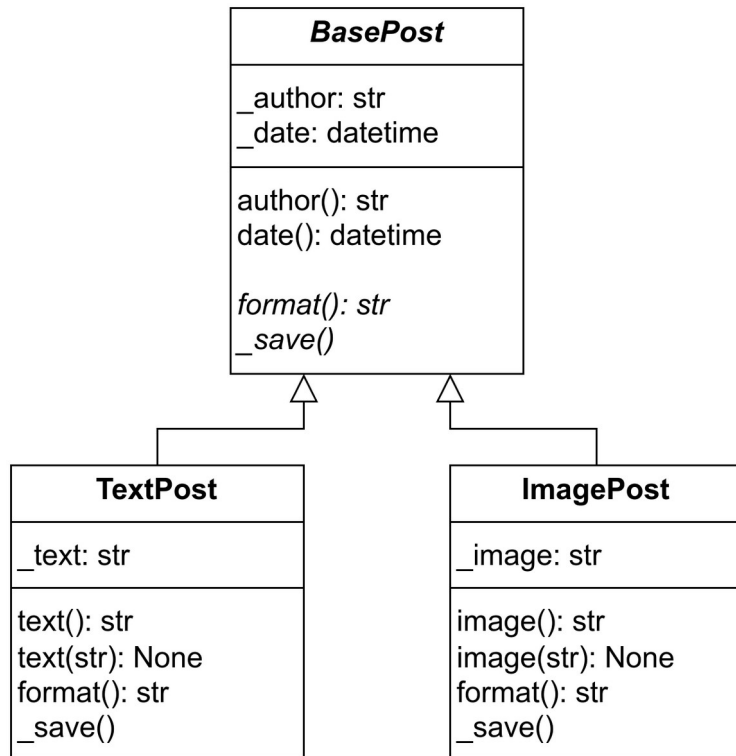
```
File ".../main.py", line 6, in <module>
```

```
    feed.append(ImagePost("Малевич К.С.", "black_square.jpg"))
```

```
~~~~~^
```

```
TypeError: Can't instantiate abstract class ImagePost without an implementation for abstract method  
'format'
```

Диаграмма с абстрактным базовым классом BasePost



```
> python main.py
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Текстовый пост сохранен.
```

```
TextPost.__init__()
```

```
BasePost.__init__()
```

```
Пост с картинкой сохранен.
```

```
=====
```

```
Автор: Толстой Л.Н.
```

```
Дата: 2025-03-23 10:13:56.335096+00:00
```

```
----
```

```
Очень длинный текст...
```

```
=====
```

```
=====
```

```
Автор: Малевич К.С.
```

```
Дата: 2025-03-23 10:13:56.335112+00:00
```

```
Картинка: black_square.jpg
```

```
=====
```

```
src/14. 00P/example_11/
```

Специальные методы класса ("магические" методы, dunder-методы)

* dunder — double underscores

"Магический" метод	Назначение
__init__	Конструктор класса.
__call__	Делает объект вызываемым, т. е. добавляет возможность применения оператора "круглые скобки".
__hash__	Делает объект хэшируемым.
__len__	Позволяет применять функцию <code>len()</code> к объекту.
__getitem__	Добавляет возможность применения оператора "квадратные скобки" для получения значения.
__setitem__	Добавляет возможность применения оператора "квадратные скобки" для изменения значения.

"Магический" метод	Назначение
<code>__add__</code>	Позволяет применять к объекту оператор суммирования "+".
<code>__sub__</code>	Позволяет применять к объекту оператор вычитания "-".
<code>__mul__</code>	Позволяет применять к объекту оператор умножения "*".
<code>__truediv__</code>	Позволяет применять к объекту оператор деления "/".
<code>__floordiv__</code>	Позволяет применять к объекту оператор целочисленного деления "//".
<code>__mod__</code>	Позволяет применять к объекту оператор вычисления остатка от деления "%".

"Магический" метод	Назначение
<code>__str__</code>	Возвращает строковое представление объекта.
<code>__repr__</code>	Возвращает строковое представление объекта при передаче его в функцию <code>repr()</code> . Иногда это более подробный формат, используемый для отладки, по сравнению с <code>__str__()</code> .
<code>__bool__</code>	Позволяет преобразовывать объект в булево значение.
<code>__int__</code>	Позволяет преобразовывать объект в целое число.
<code>__float__</code>	Позволяет преобразовывать объект в дробное число.

"Магический" метод	Назначение
<code>__lt__</code>	Позволяет применять к объекту оператор "<"
<code>__le__</code>	Позволяет применять к объекту оператор "<="
<code>__gt__</code>	Позволяет применять к объекту оператор ">"
<code>__ge__</code>	Позволяет применять к объекту оператор ">="
<code>__eq__</code>	Позволяет применять к объекту оператор "=="
<code>__ne__</code>	Позволяет применять к объекту оператор "!="
<code>__contains__</code>	Позволяет применять к объекту оператор in.


```
# vector.py
```

```
class Vector2D:
    """Класс вектора на плоскости"""
    def __init__(self, x1, y1, x2, y2):
        self.p1 = (x1, y1)
        self.p2 = (x2, y2)
```

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(x1=1, y1=1, x2=3, y2=2)
```

```
CD = Vector2D(x1=2, y1=1, x2=3, y2=5)
```

```
print("AB:", AB)
```

```
print("CD:", CD)
```

Результат выполнения скрипта

```
> python main.py
```

```
AB: <vector.Vector2D object at 0x7f3d8e737380>
```

```
CD: <vector.Vector2D object at 0x7f3d8e371810>
```

```
# vector.py
```

```
class Vector2D:
```

```
    """Класс вектора на плоскости"""
```

```
    def __init__(self, x1, y1, x2, y2):  
        self.p1 = (x1, y1)  
        self.p2 = (x2, y2)
```

```
    def __str__(self):  
        return f"{self.p1} -> {self.p2}"
```

Результат выполнения скрипта

```
> python main.py  
AB: (1, 1) -> (3, 2)  
CD: (2, 1) -> (3, 5)
```

Применение функции `abs()` к экземплярам класса `Vector2D`

45

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = Vector2D(2, 1, 3, 5)
```

```
print("AB:", AB)
```

```
print("CD:", CD)
```

```
print("|AB|:", abs(AB))
```

```
print("|CD|:", abs(CD))
```

src/14. 00P/example_14/

Применение функции `abs()` к экземплярам класса `Vector2D`

```
# vector.py
```

```
from math import hypot
```

```
class Vector2D:
```

```
    """Класс вектора на плоскости"""
```

```
    def __init__(self, x1, y1, x2, y2):  
        self.p1 = (x1, y1)  
        self.p2 = (x2, y2)
```

```
    def __str__(self):  
        return f"{self.p1} -> {self.p2}"
```

```
@property
```

```
    def coord(self):  
        return (self.p2[0] - self.p1[0], self.p2[1] - self.p1[1])
```

```
    def __abs__(self):  
        return hypot(*self.coord)
```

src/14. 00P/example_14/

Результат выполнения скрипта

```
> python main.py  
AB: (1, 1) -> (2, 2)  
CD: (2, 1) -> (3, 5)  
|AB|: 1.4142135623730951  
|CD|: 4.123105625617661
```

Сравнение экземпляров класса Vector2D

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = Vector2D(1, 1, 2, 2)
```

```
EF = Vector2D(3.0, 3.0, 4.0, 4.0)
```

```
GH = Vector2D(1, 2, 3, 4)
```

```
print("AB == CD:", AB == CD)
```

```
print("AB == EF:", AB == EF)
```

```
print("AB == GH:", AB == GH)
```

src/14. 00P/example_15/

Результат выполнения скрипта

```
> python main.py  
AB == CD: False  
AB == EF: False  
AB == GH: False
```

Сравнение экземпляров класса Vector2D

```
# vector.py
```

```
from math import hypot, isclose
```

```
class Vector2D:
```

```
    ...
```

```
    def __eq__(self, other):
```

```
        if not isinstance(other, Vector2D):
```

```
            return False
```

```
        self_coord = self.coord
```

```
        other_coord = other.coord
```

```
        return (isclose(self_coord[0], other_coord[0]) and  
                isclose(self_coord[1], other_coord[1]))
```

Результат выполнения скрипта

```
> python main.py  
AB == CD: True  
AB == EF: True  
AB == GH: False
```

Применение арифметических операторов

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 3, 2)
```

```
CD = Vector2D(2, 2, 4, 4)
```

```
print("AB + CD =", AB + CD)
```

```
print("AB - CD =", AB - CD)
```

Применение арифметических операторов

```
# vector.py
```

```
class Vector2D:
```

```
...
```

```
def __add__(self, other):
```

```
    delta = other.coord
```

```
    x2 = self.p2[0] + delta[0]
```

```
    y2 = self.p2[1] + delta[1]
```

```
    return Vector2D(self.p1[0], self.p1[1], x2, y2)
```

```
def __sub__(self, other):
```

```
    delta = other.coord
```

```
    x2 = self.p2[0] - delta[0]
```

```
    y2 = self.p2[1] - delta[1]
```

```
    return Vector2D(self.p1[0], self.p1[1], x2, y2)
```

Результат выполнения скрипта

```
> python main.py
```

```
AB + CD = (1, 1) -> (5, 4)
```

```
AB - CD = (1, 1) -> (1, 0)
```

Применение арифметических операторов с присваиванием

55

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = Vector2D(2, 2, 4, 4)
```

```
print("AB:", AB)
```

```
AB += CD
```

```
print("AB += CD")
```

```
print("AB:", AB)
```

src/14. 00P/example_18/

Результат выполнения скрипта

```
> python main.py  
AB: (1, 1) -> (2, 2)  
AB += CD  
AB: (1, 1) -> (4, 4)
```


Применение арифметических операторов с присваиванием (проверка id)

57

```
# main-2.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = Vector2D(2, 2, 4, 4)
```

```
print("AB:", AB)
```

```
print(f"{{id(AB)}}")
```

```
AB += CD
```

```
print("AB += CD")
```

```
print("AB:", AB)
```

```
print(f"{{id(AB)}}")
```

src/14. 00P/example_18/

Результат выполнения скрипта

```
> python main-2.py  
AB: (1, 1) -> (2, 2)  
id(AB)=139941372982816  
AB += CD  
AB: (1, 1) -> (4, 4)  
id(AB)=139941369026896
```

Применение арифметических операторов с присваиванием

59

```
# vector.py
```

```
class Vector2D:
```

```
...
```

```
def __iadd__(self, other):
```

```
    delta = other.coord
```

```
    x2 = self.p2[0] + delta[0]
```

```
    y2 = self.p2[1] + delta[1]
```

```
    self.p2 = (x2, y2)
```

```
    return self
```

```
def __isub__(self, other):
```

```
    delta = other.coord
```

```
    x2 = self.p2[0] - delta[0]
```

```
    y2 = self.p2[1] - delta[1]
```

```
    self.p2 = (x2, y2)
```

```
    return self
```

src/14. 00P/example_19/

Применение арифметических операторов с присваиванием

60

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = Vector2D(2, 2, 4, 4)
```

```
print("AB:", AB)
```

```
print(f"{id(AB)=}")
```

```
AB += CD
```

```
print("AB += CD")
```

```
print("AB:", AB)
```

```
print(f"{id(AB)=}")
```

src/14. 00P/example_19/

Результат выполнения скрипта

```
> python main.py  
AB: (1, 1) -> (2, 2)  
id(AB)=140161651538112  
AB += CD  
AB: (1, 1) -> (4, 4)  
id(AB)=140161651538112
```

Добавление оператора умножения

```
# vector.py
```

```
class Vector2D:
```

```
...
```

```
    def __mul__(self, n):
```

```
        coord = self.coord
```

```
        x2 = self.p1[0] + n * coord[0]
```

```
        y2 = self.p1[1] + n * coord[1]
```

```
        return Vector2D(self.p1[0], self.p1[1], x2, y2)
```

```
    def __imul__(self, n):
```

```
        coord = self.coord
```

```
        x2 = self.p1[0] + n * coord[0]
```

```
        y2 = self.p1[1] + n * coord[1]
```

```
        self.p2 = (x2, y2)
```

```
        return self
```

src/14. 00P/example_20/

Добавление оператора умножения

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = AB * 2
```

```
print("CD:", CD)
```

```
print(f"{id(AB)=}")
```

```
AB *= 3
```

```
print("AB *= 3")
```

```
print("AB:", AB)
```

```
print(f"{id(AB)=}")
```

src/14. 00P/example_20/

Результат выполнения скрипта

```
> python main.py  
CD: (1, 1) -> (3, 3)  
id(AB)=140286698420416  
AB *= 3  
AB: (1, 1) -> (4, 4)  
id(AB)=140286698420416
```


Поменяем местами множители

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 2, 2)
```

```
CD = AB * 2
```

```
print("CD:", CD)
```

```
EF = 2 * AB
```

```
print("EF:", EF)
```

Результат выполнения скрипта

```
> python main.py
CD: (1, 1) -> (3, 3)
Traceback (most recent call last):
  File ".../main.py", line 8, in <module>
    EF = 2 * AB
        ~~^~~~
TypeError: unsupported operand type(s) for *: 'int'
and 'Vector2D'
```

Добавление оператора умножения справа

```
# vector.py
```

```
class Vector2D:
```

```
    ...  
    def __mul__(self, n):  
        coord = self.coord  
        x2 = self.p1[0] + n * coord[0]  
        y2 = self.p1[1] + n * coord[1]  
        return Vector2D(self.p1[0], self.p1[1], x2, y2)  
  
    def __rmul__(self, n):  
        return self.__mul__(n)
```

src/14. 00P/example_22/

Результат выполнения скрипта

```
> python main.py  
CD: (1, 1) -> (3, 3)  
EF: (1, 1) -> (3, 3)
```

Добавление оператора @

```
# vector.py
```

```
class Vector2D:
```

```
...
```

```
def __matmul__(self, other):  
    coord_self = self.coord  
    coord_other = other.coord  
    return (coord_self[0] * coord_other[0]  
            + coord_self[1] * coord_other[1])
```

Добавление оператора @

```
# main.py
```

```
from vector import Vector2D
```

```
AB = Vector2D(1, 1, 3, 2)
```

```
CD = Vector2D(2, 2, 4, 4)
```

```
print("AB @ CD =", AB @ CD)
```

Результат выполнения скрипта

```
> python main.py  
AB @ CD = 6
```

```
src/14. 00P/example_23/
```